Subarray

907. Sum of Subarray Minimums

Medium | Topics | Companies

Given an array of integers arr, find the sum of `min(b)`, where `b` ranges over every (contiguous) subarray of `arr`. Since the answer may be large, return the answer **modulo** $10^9 + 7$.

find the contribution of each element

HINT : stand on one element , and find the next smaller element and previous smaller element , then you get the range in which the current element is the smallest , then its easy

9 8 6 3 6 8 1

for 3 on the right 3 6 8 on the left 9 8 6 3 , total combination possible = 3 x 4 = 12 , thats the contribution of 3 so ans += 3 x 12

Cleaner Interview Version

```cpp
class Solution {
public:
    static const int MOD = 1e9 + 7;

    // TC = O(n)
    // SC = O(n)

    int sumSubarrayMins(vector<int>& arr) {

        int n = arr.size();

        // nse[i] = index of Next Smaller element
        // If none exists, store n.
        vector<int> nse(n, n);

        // pse[i] = index of Previous Smaller (or equal) element
        // If none exists, store -1.
        vector<int> pse(n, -1);

        stack<int> st;

        // ---------------- Next Smaller Element ----------------
        // Pop all strictly greater elements.
        for (int i = n - 1; i >= 0; i--) {

            while (!st.empty() && arr[st.top()] > arr[i])
                st.pop();

            if (!st.empty())
                nse[i] = st.top();

            st.push(i);
        }

        while (!st.empty()) st.pop();

        // -------------- Previous Smaller or Equal --------------
        // Pop greater OR equal elements to handle duplicates.
        for (int i = 0; i < n; i++) {

            while (!st.empty() && arr[st.top()] >= arr[i])
                st.pop();

            if (!st.empty())
                pse[i] = st.top();

            st.push(i);
        }

        // Count contribution of every element as the minimum.
        long long ans = 0;

        for (int i = 0; i < n; i++) {

            long long left = i - pse[i];      // choices on left
            long long right = nse[i] - i;     // choices on right

            ans = (ans + 1LL * arr[i] * left * right) % MOD;
        }

        return ans;
    }
};
```

Duplicate Handling (Very Important)

This is one of the most common interview follow-ups.

There are **two correct ways**:

PreviousNextSmaller (`<`)Smaller or Equal (`<=`)Smaller or Equal (`<=`)Smaller (`<`)

Translated into stack conditions:

Option 1 (most common):

```
// Previous Smaller
while(arr[st.top()] >= arr[i]) pop();

// Next Smaller
while(arr[st.top()] > arr[i]) pop();
```

Option 2 (equally correct):

```
// Previous Smaller
while(arr[st.top()] > arr[i]) pop();

// Next Smaller
while(arr[st.top()] >= arr[i]) pop();
```

The key rule is: **break ties on exactly one side**, otherwise duplicate values get counted incorrectly. This is an interview detail that's frequently tested.

2104. Sum of Subarray Ranges

Solved

Medium | Topics | Companies | Hint

You are given an integer array `nums`. The **range** of a subarray of `nums` is the difference between the largest and smallest element in the subarray.

Return *the **sum of all** subarray ranges of* `nums`.

A subarray is a contiguous **non-empty** sequence of elements within an array.

ANS = sum of subarray maxi - sum of subarray minimun , JUST COPY PASTE